

Movie Theatre Ticket System

Software Requirements Specification

Version 3

Aug 3, 2026

Group #21
Robert Lopez

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Summer 2026

Revision History

Date	Description	Author	Comments
07/12/22026	Version 1	Robert Lopez	Initial document creation
07/19/22026	Version 2	Robert Lopez	Added diagrams & GitHub
07/28/2026	Version 3	Robert Lopez	Added test cases
08/03/2026	Version 4	Robert Lopez	Added Section 4.1

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date

Table of Contents

REVISION HISTORY.....	II
DOCUMENT APPROVAL.....	II
1. INTRODUCTION.....	1
1.1 PURPOSE.....	1
1.2 SCOPE.....	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS.....	1
1.4 REFERENCES.....	1
1.5 OVERVIEW.....	1
2. GENERAL DESCRIPTION.....	2
2.1 PRODUCT PERSPECTIVE.....	2
2.2 PRODUCT FUNCTIONS.....	2
2.3 USER CHARACTERISTICS.....	2
2.4 GENERAL CONSTRAINTS.....	2
2.5 ASSUMPTIONS AND DEPENDENCIES.....	2
3. SPECIFIC REQUIREMENTS.....	2
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	3
3.1.1 <i>User Interfaces</i>	3
3.1.2 <i>Hardware Interfaces</i>	3
3.1.3 <i>Software Interfaces</i>	3
3.1.4 <i>Communications Interfaces</i>	3
3.2 FUNCTIONAL REQUIREMENTS.....	3
3.2.1 <i><Functional Requirement or Feature #1></i>	3
3.2.2 <i><Functional Requirement or Feature #2></i>	3
3.3 USE CASES.....	3
3.3.1 <i>Use Case #1</i>	3
3.3.2 <i>Use Case #2</i>	3
3.4 CLASSES / OBJECTS.....	3
3.4.1 <i><Class / Object #1></i>	3
3.4.2 <i><Class / Object #2></i>	3
3.5 NON-FUNCTIONAL REQUIREMENTS.....	4
3.5.1 <i>Performance</i>	4
3.5.2 <i>Reliability</i>	4
3.5.3 <i>Availability</i>	4
3.5.4 <i>Security</i>	4
3.5.5 <i>Maintainability</i>	4
3.5.6 <i>Portability</i>	4
3.6 INVERSE REQUIREMENTS.....	4
3.7 DESIGN CONSTRAINTS.....	4
3.8 LOGICAL DATABASE REQUIREMENTS.....	4
3.9 OTHER REQUIREMENTS.....	4
4. ANALYSIS MODELS.....	4
4.1 SEQUENCE DIAGRAMS.....	5
4.3 DATA FLOW DIAGRAMS (DFD).....	5
4.2 STATE-TRANSITION DIAGRAMS (STD).....	5
5. CHANGE MANAGEMENT PROCESS.....	5
A. APPENDICES.....	5
A.1 APPENDIX 1.....	5
A.2 APPENDIX 2.....	5

1. Introduction

This SRS describes the requirements for the Movie Theater Ticketing System and explains what the system needs to accomplish, the limits it needs to follow, and the information developers will use to design, build, and test it.

1.1 Purpose

To describe the requirements for the Movie Theater Ticketing System. It's written for the team members who will design, build, and maintain the system.

1.2 Scope

The Movie Theater Ticketing System will let users choose showings, select and reserve specific seats, buy up to 20 tickets per transaction and pay with a credit card, PayPal, or Bitcoin. It should also support loyalty accounts, returns and exchanges before a showing, multiple languages and currencies, high volume queues, and secure unique tickets. The system should focus on ticket sales and account management only.

1.3 Definitions, Acronyms, and Abbreviations

NFT: A unique digital token used to prevent tickets from being copied or reused.

Showing: A movie screening at a specific theater, date, and time.

Loyalty Account: An account that stores customer info, payment info, purchase history, and loyalty points.

Concurrent Users: Users accessing the system at the same time.

API: A connection that allows the system to exchange information with outside services, such as payment and currency providers.

1.4 References

- Theater Ticketing System Requirements, course-provided requirements list, 2026.
- Software Requirements Specification Template, CS 250 course document.

1.5 Overview

The rest of this SRS describes the general system, its users and constraints, the functional and non-functional requirements, the UI, and the models we'll use and explain how the Movie Theater Ticketing System works.

2. General Description

This section gives a general overview of the Movie Theater Ticketing System. Describing the functions, users, constraints and assumptions. It provides context for the detailed requirements listed later in the document.

2.1 Product Perspective

The Movie Theater Ticketing System will be a standalone platform for browsing movies, picking seats, buying tickets and managing user accounts. It should also connect with payment services, currency conversion tools and theater scheduling data.

2.2 Product Functions

The Movie Theater Ticketing System should let users browse shows, pick seats, reserve and buy tickets, manage accounts, and return/exchange tickets. It should also handle payments, currency conversion, multiple languages, receipts, large queues, and unique digital tickets.

2.3 User Characteristics

The main users are movie customers. Users can purchase as guests or create accounts, and they can access the system in English, Spanish, or another language.

2.4 General Constraints

Transactions are limited to 20 tickets, and only one device may be logged into an account at a time. The system should support up to 10 million concurrent users, accepted payment methods, local currencies, and secure non-replicable tickets.

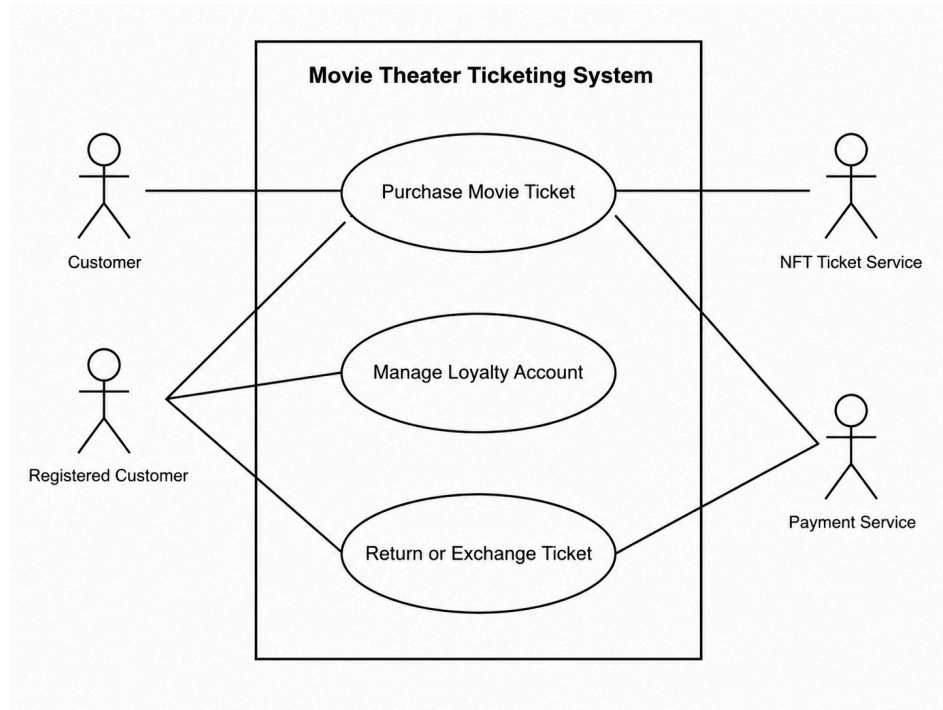
2.5 Assumptions and Dependencies

The system depends on payment providers, current currency exchange rates, theater shows and seat information, and the service used to create NFT tickets. It also assumes users have a reliable internet connection and provide valid payment and account information.

3. Specific Requirements

This section lists the detailed requirements for the Movie Theater Ticketing System. These requirements should guide the development and testing of the system.

Movie Theatre Ticket System



3.1 External Interface Requirements

The system will use several interfaces so users can browse movies, choose seats, make payments, and manage their accounts. It will also connect with outside services needed for payments, currency conversion, and ticket security.

3.1.1 User Interfaces

Users will interact with the system through a website or mobile app. The interface should include movie lists, a seat selection screen, checkout, account settings, loyalty information, and language options.

3.1.2 Hardware Interfaces

The system should support computers, phones, tablets, kiosks, and ticket scanning devices. Users will need an internet connection to access the system.

3.1.3 Software Interfaces

The system should connect with credit card, PayPal, and Bitcoin payment services. It should also use currency conversion data, theater showing and seat information, and a service that creates and verifies NFT tickets.

3.1.4 Communications Interfaces

The system should communicate over the internet using secure connections. It should use APIs to exchange information with payment providers, currency services, theater databases, and ticket services.

3.2 Functional Requirements

The Movie Theater Ticketing System should allow users to browse showings, pick seats, buy tickets and manage accounts.

3.2.1 Ticket Selection and Purchase

3.2.1.1 Introduction

- Users can choose a movie, reserve seats, and buy up to 20 tickets in one transaction.

3.2.1.2 Inputs

- Movie, showing, seat, number of tickets, and payment method.

3.2.1.3 Processing

- Check seat availability, apply the ticket price and currency, process payment, and reserve seats.

3.2.1.4 Outputs

- The user receives a purchase confirmation and a unique digital ticket for each seat.

3.2.1.5 Error Handling

- The system displays an error if seats aren't available, payment declines, or the user selects more than 20 tickets.

3.2.2 User and Loyalty Accounts

3.2.2.1 Introduction

- Users can create loyalty accounts and use them to manage their personal information and purchases.

3.2.2.2 Inputs

- Inputs include personal information, login details, payment info, and preferences.

3.2.2.3 Processing

- The system stores the user's info, receipts, payment info, and loyalty rewards or points. Only one device can be logged into the account at a time.

3.2.2.4 Outputs

- The user can view account information, receipts, saved payment methods, and loyalty points.

3.2.2.5 Error Handling

- The system displays an error for invalid login information, missing account fields, or an account already active on another device.

3.2.3 User and Loyalty Accounts

3.2.3.1 Introduction

- Users can return or exchange tickets before the movie.

3.2.3.2 Inputs

- Inputs include the user account, ticket, receipts, and requested returns or exchanges.

3.2.3.3 Processing

- The system confirms that the ticket was purchased through the account and that the showing hasn't started.

3.2.3.4 Outputs

- The user receives confirmation of the return, refund, or exchanged ticket.

3.2.3.5 Error Handling

- The request is rejected if the ticket was purchased without an account or the showing has already started.

3.3 Use Cases

3.3.1 Purchase Movie Ticket

Actor: Customer

Entry Conditions: The customer has internet access and the selected showing is available.

Main Flow:

- Customer selects a movie and showing
- Customer selects up to 20 available seats
- System displays the total in local currency
- customer chooses a payment method
- System processes the payment
- System creates unique NFT tickets and sends confirmation

Alternate Flow:

- Customer enters a queue if traffic is high
- Unavailable seats must be replaced
- Orders over 20 tickets are rejected
- Failed payments cancel the reservation

3.3.2 Manage Loyalty Account

Actor: *Registered* Customer

Entry Conditions: Customer has a loyalty account.

Main Flow:

- Customer logs in
- System confirms the account is not active on another device
- Customer views or updates account information
- Customer views purchase history and loyalty points
- System saves any changes

Alternate Flow:

- Invalid login information is rejected
- Login is blocked if another device is already using the account

3.3.3 Return or Exchange Ticket

Actor: Registered Customer

Entry Conditions: The ticket was purchased through the account and the showing hasn't started.

Main Flow:

- Customer logs in and selects a ticket
- Customer chooses return or exchange

- System checks that the ticket is eligible
- System processes the refund or exchange
- System updates the ticket and seat status
- Customer receives confirmation

Alternate Flow:

- Guest purchases are not eligible.
- Requests after the showing begins are rejected.
- Exchanges are rejected if the new seat is unavailable.

3.4 Classes / Objects

3.4.1 User Account

3.4.1.1 Attributes

- Name, contact information, login details, payment information, purchase history, and loyalty points.

3.4.1.2 Functions

- Register, log in, log out, update account information, view movie ticket history and manage payment information.

3.4.2 Showing

3.4.2.1 Attributes

- Name, contact information, login details, payment info, purchase history, and loyalty points.

3.4.2.2 Functions

- Register, log in, log out, update account information, view purchase history, and manage payment information.

3.4.3 Seat

3.4.3.1 Attributes

- Seat number, section, showing, price, and availability status.

3.4.3.2 Functions

- Reserve, release, and confirm the seat as purchased.

3.4.4 Transaction

3.4.4.1 Attributes

- User, selected tickets, total price, payment method, currency, date, and transaction status.

3.4.4.2 Functions

- Calculate the total, process payment, record the purchase, and handle refunds or exchanges.

3.4.5 Ticket

3.4.5.1 Attributes

- Ticket ID, NFT information, showing, seat, owner, and ticket status.

3.4.5.2 Functions

- Create, verify, return, exchange, and mark the ticket as used.

3.5 Non-Functional Requirements

3.5.1 Performance

- The system must support up to 10 million concurrent users. A queue will control high traffic for popular showings, and each transaction will be limited to 20 tickets.

3.5.2 Reliability

- Ticket prices and seat availability must remain consistent across the system. The system must prevent the same seat or ticket from being sold more than once.

3.5.3 Availability

- The system should maintain close to 100% availability during ticket sale periods and recover from service interruptions within five minutes without losing purchase info.

3.5.4 Security

- Payment and personal info must be protected through secure connections and access controls. Only one device can use an account at a time, and every ticket must be unique.

3.5.5 Maintainability

- The system should be organized so developers can update payment methods, languages, currency services, and other features without rebuilding the entire thing.

3.5.6 Portability

- The system should work on desktop/laptop computers, phones, tablets, kiosks, and on web browsers.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

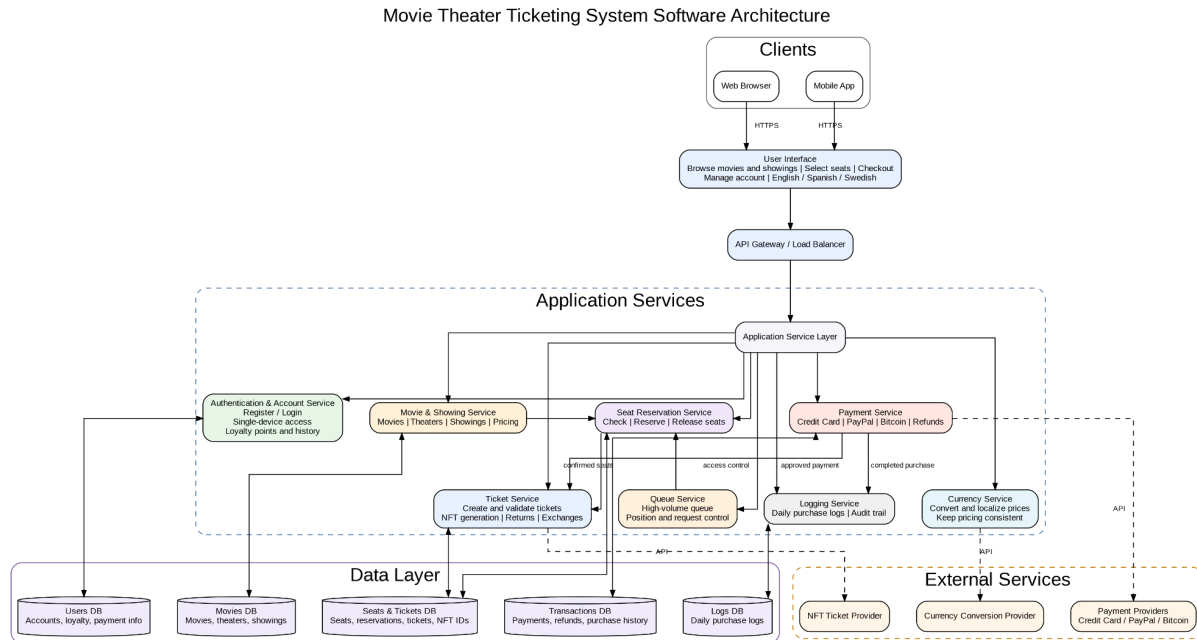
4. Analysis Models

Here we describe the planned organization and implementation of the Movie Theater Ticketing System. It'll show the system architecture, major classes, class relationships, and the development plan that will be followed by the team.

Movie Theatre Ticket System

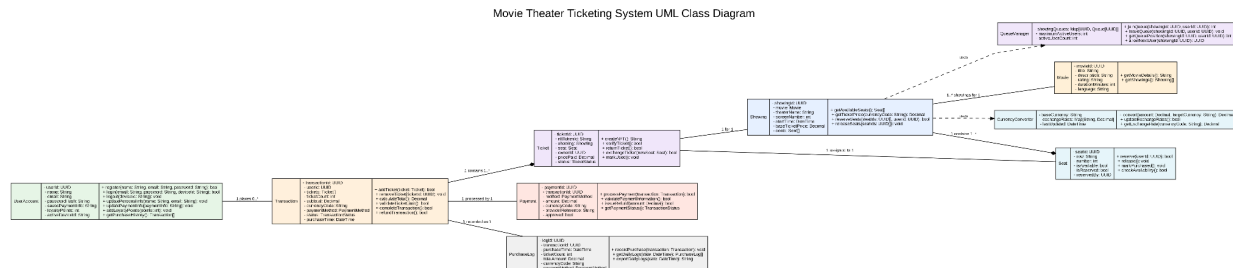
The following are the software architecture and UML class diagrams. Not really sure where to put these right now.

Movie Theater Ticketing System Software Architecture:



The architecture diagram shows how the user interface connects to the system services, databases, payment providers, currency service, and NFT provider.

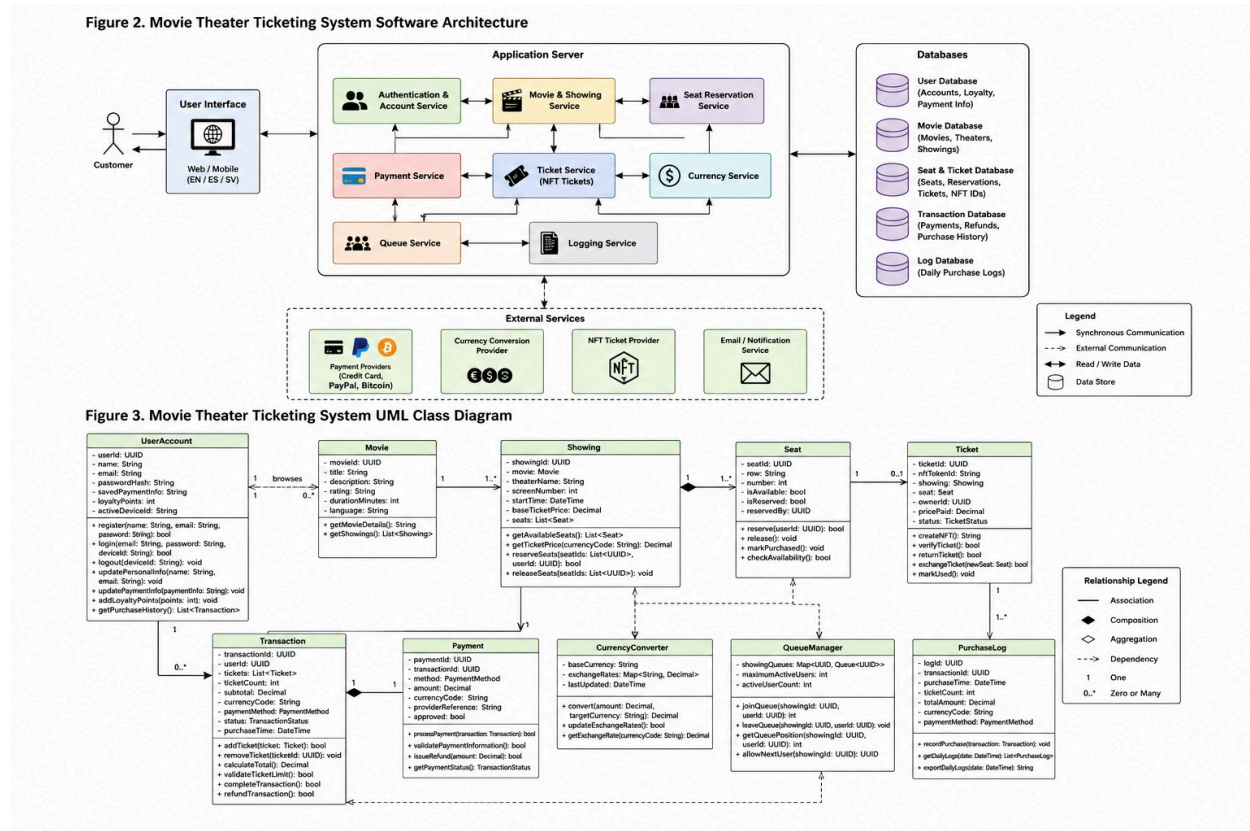
Figure 3. Movie Theater Ticketing System UML Class Diagram:



The UML diagram shows the main classes, attributes, operations, datatypes, parameters, and relationships. The classes include UserAccount, Movie, Showing, Seat, Ticket, Transaction, Payment, CurrencyConverter, QueueManager, and PurchaseLog.

4.1 Architecture Design

Design Specification



Data Management Strategy

The Movie Theater Ticketing System will use a relational SQL database, such as PostgreSQL, since the system has connected data like users, showings, seats, tickets, and transactions.

The data is divided into five logical databases:

1. Users Database stores account information, loyalty points, purchase history, and saved payment references.
2. Movies Database stores movies, theaters, showings, times, and prices.
3. Seats and Tickets Database stores seat availability, reservations, tickets, and NFT IDs.
4. Transactions Database stores purchases, payments, refunds, and transaction statuses.
5. Logs Database stores the systemwide daily purchase logs.

These could start as separate schemas on the same database server and be split into separate servers later if the system grows. This keeps the first version easier to manage while still allowing the system to scale.

Passwords should be hashed, and full payment information won't be stored directly. Payment providers will return secure references or tokens instead. Database access should be limited by service, and backups will be created regularly.

Ticket purchases will use database transactions so seat reservations, payments, and ticket creation either finish together or are rolled back. This should help prevent duplicate ticket sales and incomplete purchases.

Tradeoff Discussion

A single database would be easier to build and maintain, but it would be harder to scale and would place all sensitive data in one location. Splitting the data logically adds some complexity, but it improves organization, security, and scalability.

A NoSQL database was also considered since it can handle large amounts of traffic and flexible data. SQL is a better fit since ticket purchases depend on strong relationships, consistent prices, and reliable transactions. The main tradeoff is that SQL may require more planning when scaling, but it provides stronger data integrity for seat reservations and payments.

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

6.6 Test Cases

6.1 Purpose

This test plan explains how the Movie Theater Ticketing System will be checked before release. The tests cover ticket limits, seat reservations, payments, account access, returns, currency conversion, queues, NFT tickets, and purchase logs. Current architecture and UML class diagram are still valid, so no design changes are needed right now.

6.2 Test Environment

Testing will use test accounts, movies, showings, seats, and payment information. Payment, currency, and NFT services will use test or mock versions so no real money or tickets are created.

6.3 Unit Testing

The plan includes three unit tests that check individual methods and classes, including the 20 ticket limit, currency conversion, and seat availability.

6.4 Functional Testing

The plan includes four functional tests that check complete features, including ticket purchases, failed payments, account access, and ticket returns or exchanges.

6.5 System Testing

The plan includes three system tests that check the full system, including high traffic queues, duplicate seat prevention, and purchases using language, currency, payment, NFT, and logging services.

6.6 Test Cases

The Excel file contains 10 sample test cases:

3 unit tests

4 functional tests

3 system tests

Each test case includes its prerequisites, steps, expected result, actual result, status, and tester.

6.7 Test Completion

Testing is complete when all 10 test cases have been run, important tests pass, and no open issue could cause duplicate seat sales or lost payment information. When a test fails, the result will be recorded and a GitHub issue will be created. The test will be run again after the issue is fixed.

6.8 GitHub Link

<https://github.com/rlopez4042/cs--50-project>

6.9 Responsibility

Robert Lopez is the only group member and is responsible for the test plan, test cases, GitHub commit, issue tracking, and final SRS update.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2